

Phase 1 - System-on-Chip (SoC) Design

This practical consisted of two parts focused at introducing FPGA based system-on-chip (SoC) design using Quartus and Qsys tools. In Part 1, we developed a simple counter system to understand the basic hardware and software workflow. In Part 2, we extended this design to implement a JPEG encoder, which introduced several new concepts like external memory, clock handling, and clock domain crossing.

Part 1: First SoC - LED Counter

In Part 1 of the lab, we designed a basic SoC using Quartus II and Qsys, which included a Nios II processor, on-chip memory, a timer, a PIO module connected to LEDs, JTAG UART, and a system ID component. We created a top-level hardware file (TopLevel.bdf) to connect the SoC to the FPGA board, including the clock input and LED outputs.

After compiling the hardware design and programming the FPGA, we wrote a simple C program that used the processor to control the LEDs as a counter. For this part, we used on-chip memory because the application was small and did not require a large amount of storage. We also used JTAG UART for input and output, since it allowed us to communicate with the program using the same cable used to program the FPGA.

Through this part, we understood the complete FPGA design flow, including creating hardware in Qsys, integrating it in Quartus, mapping pins and running software on the processor. It also introduced us to the concept of memory-mapped peripherals.

Part 2: JPEG Encoder

In Part 2, we extended the same design flow to implement a JPEG encoder system. Compared to Part 1, this design was more complex due to the use of external SDRAM, added a PLL to generate multiple clocks and included a Clock Crossing Bridge to handle communication between components running at different clock speeds.

The software application reads BMP image files from the PC using the JTAG interface, converts them into JPEG format, and saves the output back to the PC. We also modified the code to display different processing stages using specific hexadecimal values mapped to the PIO LED outputs.

State	Hex Value	State	Hex Value	State	Hex Value
IDLE	0x00	EXTRACTING	0x03	DONE	0xFF
VALID_BMP	0x01	ENCODING	0x07	ERROR	0xAA

External SDRAM was used instead of on-chip memory because JPEG encoding requires large memory buffers, which cannot fit into on-chip memory. Therefore, external SDRAM was used as the main memory for the system. The SDRAM base address was explicitly set to 0x00000000 so that the processor could boot and run directly from external memory.

The FPGA board provides only a single 50 MHz clock. However, our system required different clock speeds. The PLL was used to generate a 100 MHz clock for the CPU, a 10 MHz clock for slower peripherals, and a phase-shifted clock for the SDRAM. The phase-shifted clock ensures that data signals are stable when the SDRAM reads them. This is necessary because SDRAM is an external component and is sensitive to timing issues.

We used 'Debug As' instead of 'Run As' because the application relies on JTAG-based file access and external memory. Debug mode ensures proper initialization and stable execution.

Problems Faced and How We Solved Them

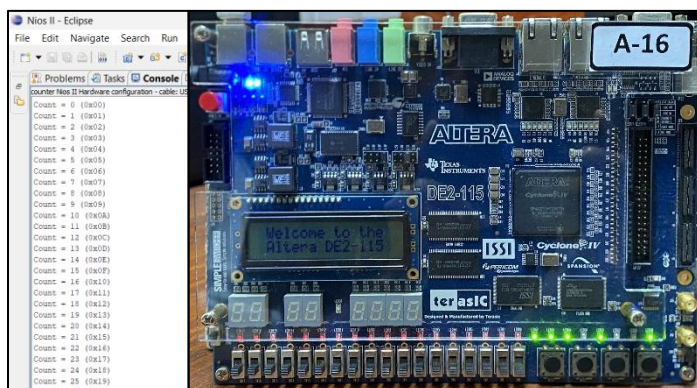
One major issue we faced was related to CPU reset and exception errors, where the processor did not know where to start execution. We fixed this by correctly setting the reset and exception vectors to the SDRAM controller.

Another challenge was dealing with PLL warnings related to unused signals. After understanding that these signals were optional for our design, we safely ignored them.

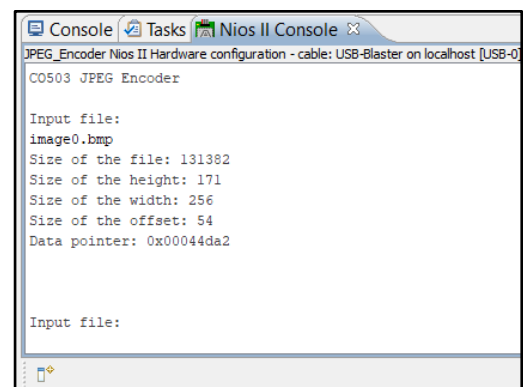
Initially, the LED status indicators were not visible long enough to be noticed. We solved this by adding small delays in the software so that completion and error states could be clearly observed.

What We Learned

This lab helped us gain practical experience in FPGA-based SoC design and hardware-software integration. We learned how external memory is used in real systems, why clock management is important and how improper configuration can lead to system failures. And more importantly, we learned how to debug both hardware and software issues systematically.



Counter SoC



JPEG Encoder Soc